

Journée 1

Architecture Micro-services

Nom / Prénom : DANILO Elouan

1. Découpage en services

Le système retenu est une plateforme de livraison de repas à domicile. Le **domaine** consiste à mettre en relation trois acteurs (clients, restaurants et livreurs) pour commander un repas, le payer, le faire préparer puis le livrer. Conformément à la règle d'or de la S2, on découpe par **capacité métier** (ce que le métier fait), jamais par couche technique.

Proposition de découpage en 6 services

Service	Capacité métier	Données possédées
1. Comptes & Identité	Inscrire, authentifier et gérer les profils (clients, restaurants, livreurs) et les droits d'accès.	Utilisateurs, rôles, adresses, jetons.
2. Restaurants & Menus	Présenter l'offre : fiches restaurants, menus, plats, prix, disponibilités et horaires d'ouverture.	Restaurants, menus, plats, catégories.
3. Commandes	Gérer le cycle de vie d'une commande : panier, validation, suivi des états (créée → confirmée → préparée → livrée).	Commandes, lignes de commande, états.
4. Paiement	Encaisser, autoriser et rembourser les transactions. Isolé par criticité (audit, sécurité, conformité PCI-DSS).	Transactions, remboursements (réfs).
5. Livraison	Affecter un livreur, suivre la position GPS, estimer le temps d'arrivée (ETA) et clôturer la course.	Courses, affectations, positions, ETA.
6. Notifications	Informé chaque acteur en temps réel : push, SMS, email transactionnels (commande confirmée, livreur en route...).	Modèles de messages, journaux d'envoi.

Bounded contexts (DDD)

Chaque service correspond à un *bounded context*, c'est-à-dire une frontière à l'intérieur de laquelle un mot a **un** sens précis. L'exemple le plus parlant ici est le mot *Commande*, qui n'a pas le même modèle selon le contexte :

- Dans le contexte **Commandes** (vente), une commande regroupe les plats choisis, le prix, le client et le mode de paiement. États : panier → confirmée → payée.
- Dans le contexte **Livraison** (logistique), une *commande* est une course avec une adresse, une position de livreur, un ETA et un numéro de suivi. États : affectée → récupérée → livrée.
- **Même mot, deux modèles distincts.** Le DDD valide d'avoir deux bounded contexts. Chaque équipe parle son *ubiquitous language* (ex. *course* côté Livraison).

D'autres frontières sont nettes. Un *Restaurant* côté Restaurants & Menus (offre, plats) n'est pas le même objet que côté Paiement (un bénéficiaire de versement). Le mot *Client* appartient au contexte Comptes & Identité. Les autres services n'en gardent qu'une référence (l'identifiant).

Anti-patterns évités (contrôle S2)

- **Pas de découpage par couche technique.** Aucun *service UI* ni *service base de données*. Chaque service est une tranche métier verticale.
- **Pas de shared database.** Chaque service possède sa propre base. Les autres passent par l'API, jamais par la table directement.

-
- **Pas de distributed monolith ni de chatty service.** La confirmation d'une commande n'appelle pas 6 services en synchrone. Le service Commandes publie un événement `CommandeConfirmée` `z` que Livraison et Notifications consomment (cohérence à terme, *eventual consistency*).
 - **Pas de god service.** Aucun service central ne pilote tous les autres, pas de hub unique.
 - **Pas d'entity service.** On a un service `Commandes` `z` (capacité), pas un service par table SQL.
 - **Paiement isolé par criticité,** un choix volontaire pour la sécurité et l'audit.

2. Conception de l'API REST

Le service retenu est le service **Commandes**, le cur transactionnel de la plateforme. L'API suit le style REST (ressources au pluriel, verbes HTTP, codes de statut explicites) et est versionnée par URI (`/api/v1/`).

Endpoints REST (URI + verbe + code de retour)

Verbe	URI	Action	Code
GET	<code>/api/v1/orders/</code>	Liste paginée des commandes (filtrable).	200
POST	<code>/api/v1/orders/</code>	Créer une commande.	201
GET	<code>/api/v1/orders/{id}/</code>	Détail d'une commande.	200 / 404
PATCH	<code>/api/v1/orders/{id}/</code>	Modifier partiellement (ex. adresse).	200
GET	<code>/api/v1/orders/{id}/items</code>	Lignes (plats) de la commande.	200
POST	<code>/api/v1/orders/{id}/cancel</code>	Annuler en créant une ressource d'annulation plutôt qu'un verbe <code>/cancelOrder</code> .	201 / 409
GET	<code>/api/v1/orders/{id}/track</code>	État de suivi (délégué à Livraison).	200

Par choix de modélisation, il n'y a pas de DELETE sur une commande. Une commande payée est un fait comptable. On ne la supprime pas, on l'**annule** (changement d'état via la ressource `cancellation`). On pense ressources et états, pas actions.

Exemple de payload JSON (création, POST `/api/v1/orders/`)

```
{
  "customer_id": 42,
  "restaurant_id": 7,
  "delivery_address": "12 rue des Lilas, 75011 Paris",
  "items": [
    { "dish_id": 101, "quantity": 2 },
    { "dish_id": 205, "quantity": 1 }
  ],
  "payment_method": "card"
}
```

La réponse 201 Created contient la commande créée avec son id, son état initial (n created z), le total calculé côté serveur (jamais envoyé par le client) et les horodatages `created_at` / `updated_at`.

Pagination, filtrage, tri, versioning

- Le **versioning** se fait par URI (`/api/v1/...`). C'est la stratégie la plus lisible et debuggable. L'ancienne version est maintenue 6 à 12 mois en cas de v2.
- La **pagination** utilise `PageNumberPagination` (`?page=2&page_size=20`) avec la réponse standard `{ count, next, previous, results }`. Pour de gros volumes (historique), on basculerait sur une pagination *cursor-based* (performance constante).
- **Filtrage** par paramètres : `?status=confirmed, ?restaurant=7, ?customer=42`.
- Le **tri** se fait avec `?ordering=-created_at` (le préfixe - signifie décroissant).
- Les **erreurs structurées** suivent le format RFC 7807 (`application/problem+json`) avec `title`, `detail` et le détail des champs invalides. Jamais de stack trace en production.

En cas d'erreur de validation, l'API renvoie par exemple un 422 avec un corps `application/problem+json` indiquant que le champ n items z ne peut pas être vide, avec un code de statut 4xx (et non 200).

3. Retour sur le TP (service Catalogue)

Le TP de la S4 a consisté à coder le **premier** micro-service de FlexShop, le service Catalogue, une API REST CRUD sur la ressource `Product`, avec Django + Django REST Framework, drf-spectacular (OpenAPI) et django-filter.

Documentation OpenAPI : Swagger UI fonctionnel

La documentation interactive est générée automatiquement par drf-spectacular et servie sur `/api/docs/`. Elle expose les 7 opérations (CRUD complet + endpoint bonus `low-stock`) et les schémas (`Product`, `PaginatedProductList`, `PatchedProduct`).

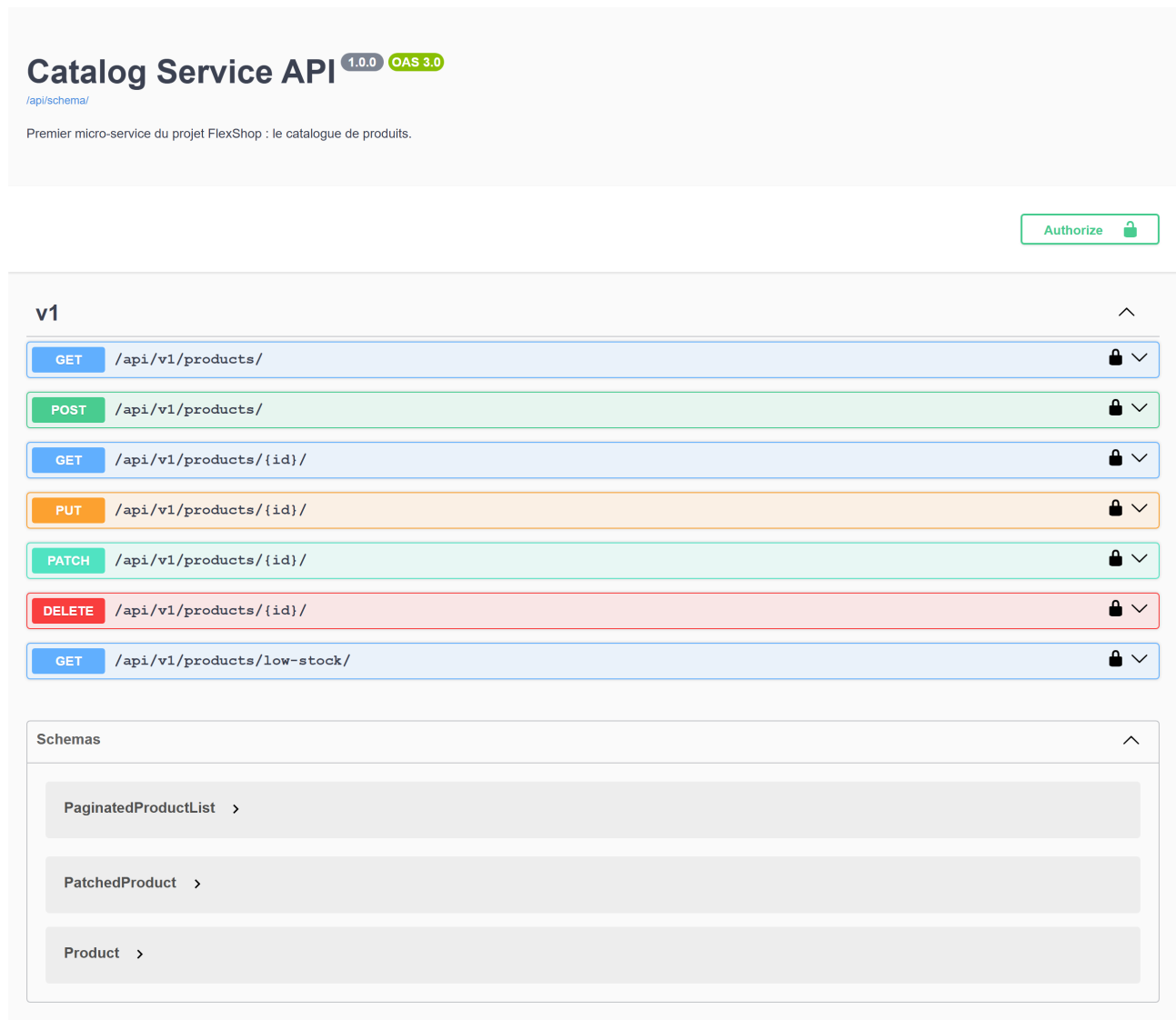


FIGURE 1 – Swagger UI du service Catalogue (<http://localhost:8000/api/docs/>).

Collection Postman (`CatalogService.postman_collection.json`)

La collection **Catalog Service – FlexShop** fournie avec le projet regroupe **10 requêtes** prêtes à l'emploi, couvrant tout le CRUD ainsi que le filtrage, la recherche, le tri et le bonus :

- GET Lister les produits · GET Détail produit · POST Créer un produit
- PATCH Modifier partiellement · PUT Remplacer · DELETE Supprimer
- GET Recherche (`?search=casque`) · GET Tri par prix décroissant (`?ordering=-price`) · GET Filtre par catégorie (`?category=`)

— GET Bonus – stock faible (/products/low-stock/)

Extraits de code marquants

Le modèle Product le prix en DecimalField avec validateur (jamais FloatField) :

```
class Product(models.Model):
    name = models.CharField(max_length=200)
    description = models.TextField(blank=True, default='')
    price = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0)], # prix >= 0
    )
    stock = models.PositiveIntegerField(default=0)
    category = models.CharField(max_length=50, blank=True, default='')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Le ViewSet tout le CRUD + les 3 backends de filtre + l'endpoint bonus @action :

```
class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_fields = ['category'] # ?category=electronics
    search_fields = ['name', 'description'] # ?search=casque
    ordering_fields = ['price', 'created_at', 'stock']

    @action(detail=False, methods=['get'], url_path='low-stock')
    def low_stock(self, request): # bonus : stock < 5
        produits = self.get_queryset().filter(stock__lt=5)
        ...
```

Tests unitaires (6 tests, tous au vert)

Les tests couvrent : liste, création (201), 404, prix négatif rejeté (400), suppression (204) et recherche.

```
$ python manage.py test products
Ran 6 tests in 0.071s

OK
```

Choix techniques notables

- Montants en DecimalField (max_digits=10, decimal_places=2) et **jamais** FloatField (les arrondis binaires sont une faute éliminatoire). On stocke 89.90 directement, ce qui reste lisible dans le JSON. L'alternative fintech (Stripe/PayPal) serait un IntegerField en centimes (8990).
- Validation métier dans le serializer avec validate_price (prix ≥ 0) et validate_name (nom non vide). DRF renvoie alors un 400 précis sur le champ fautif.
- ModelViewSet + DefaultRouter donnent tout le CRUD (list, retrieve, create, update, partial_update, destroy) en moins de 20 lignes utiles. C'est la puissance de DRF.
- Filtrage, recherche et tri activés via les trois backends : DjangoFilterBackend (?category=), SearchFilter (?search= sur name + description), OrderingFilter (?ordering=-price).
- Pagination par défaut avec PageNumberPagination et PAGE_SIZE = 10, soit une réponse { count, next, previous, results }.
- Versioning par URI (/api/v1/products/).
- Documentation OpenAPI 3 générée automatiquement par drf-spectacular (Swagger UI sur /api/docs/, schéma exporté dans openapi-schema.yml), avec une collection Postman fournie.

-
- En bonus, un endpoint custom @action GET /api/v1/products/low-stock/ liste les produits dont le stock est < 5 (réapprovisionnement à prévoir).

Difficultés rencontrées et solutions

Difficulté	Solution
ModuleNotFoundError au lancement.	venv non activé. Réactiver (venv\Scripts\activate sous Windows) et vérifier le prompt (venv).
Erreur n° App 'products' isn't installed.	Ajouter 'products' dans INSTALLED_APPS.
Changements du modèle non pris en compte.	Après toute modif de models.py, lancer makemigrations puis migrate.
POST renvoyait 415 Unsupported Media Type.	Dans Postman, régler Body → raw → JSON (et non Text).
POST renvoyait 400 sans raison claire.	Lire le corps de la réponse. DRF indique précisément le champ en erreur.
Risque de committer db.sqlite3 et venv/.	Écrire le .gitignore avant le premier ajout (venv/, __pycache__/, *.pyc, db.sqlite3, .env).

Ce qui resterait à améliorer pour aller en production

- **Base de données.** Remplacer SQLite par PostgreSQL, sortir SECRET_KEY et DEBUG dans des variables d'environnement (.env) et passer DEBUG=False.
- **Sécurité.** Authentification et autorisation (JWT), permissions par rôle, CORS restreint, rate limiting.
- **Robustesse.** Erreurs au format RFC 7807, gestion fine des conflits (409), healthcheck (/health) pour l'orchestrateur.
- **Performance.** Pagination cursor-based pour un très grand catalogue, index sur les champs filtrés et recherchés.
- **Exploitation (DevOps).** Conteneurisation Docker, pipeline CI/CD (tests + lint automatiques), observabilité (logs structurés, métriques, tracing distribué).
- **Tests.** Étendre la couverture (PUT/PATCH, filtres, pagination, cas limites) au-delà des 6 tests actuels.